

Highlights

DSN: Energy-Efficient EMG Signal Classification Method Enabling Real-Time Monitoring for Edge Healthcare

Zhenyu Zhang, Xianzhe Meng, Jianglan Wei, Mingjie Zeng, Zhigang Zeng

- We develop a highly competitive three-class fatigue classifier through the synergistic integration of SNN and HDC. With an average accuracy of 94.40%, the classifier meets the demands of AI-driven remote healthcare applications.
- The DSN Framework features ultra-low energy consumption enabled by randomized weight design and lightweight one-step training of its SNN component, which strongly supports long-term continuous monitoring on wearable devices.
- The framework demonstrates strong robustness to environmental noise and excellent adaptability to small-sample scenarios. These characteristics effectively overcome data scarcity challenges commonly encountered in personalized healthcare.

DSN: Energy-Efficient EMG Signal Classification Method Enabling Real-Time Monitoring for Edge Healthcare

Zhenyu Zhang^{a,b,1}, Xianzhe Meng^{a,1}, Jianglan Wei^a, Mingjie Zeng^a, Zhigang Zeng^{a,b,*}

^a*School of Artificial Intelligence and Automation, Huazhong University of Science and Technology, Wuhan, 430074, China*

^b*Key Laboratory of Image Processing and Intelligent Control of Education Ministry of China, Wuhan, 430074, China*

Abstract

Surface electromyography (sEMG) signal classification is challenged by high computational cost and energy consumption, which limit its deployment on resource-constrained wearable devices. To address these issues, we propose DSN, a hybrid framework that combines Spiking Neural Networks (SNNs) with Hyper-dimensional Computing (HDC). The SNN module adopts randomized initialized weights with lightweight one-step training for efficient feature extraction, while the HDC module leverages high-dimensional distributed representations for robust classification. Based on theoretical derivation, we've proved its noise robustness, efficiency and hardware error robustness. Experimental results show that our method achieves energy consumption up to $26.87\times$ lower compared with LSTM and $42.42\times$ lower compared with GRU, while maintaining over 90% accuracy using only 20% of the training data. Moreover, the framework exhibits strong robustness to noise and hardware faults, making it highly suitable for real-time, energy-efficient sEMG monitoring on edge healthcare platforms.

Keywords: EMG Signal Classification, Spiking Neural Networks,

*Corresponding author

Email addresses: zzyzhang@hust.edu.cn (Zhenyu Zhang),
U202410203@hust.edu.cn (Xianzhe Meng), U202215062@hust.edu.cn (Jianglan Wei),
U202215178@hust.edu.cn (Mingjie Zeng), zgzung@hust.edu.cn (Zhigang Zeng)

¹The two authors contributed equally to this work.

1. Introduction

Surface electromyography (sEMG) is a non-invasive technique that measures the electrical activity of muscle motor units. It provides real-time data, supports multi-channel acquisition, and is well-suited for integration with wearable sensor systems[1, 2]. Despite these advantages, applying sEMG signal classification in practical edge devices faces major challenges, primarily due to high computational demands and energy consumption. These limitations, combined with the restricted battery capacity of wearable devices, hinder their long-term deployment.

Neuromorphic computing paradigms offer promising solutions to these challenges. Among them, Spiking Neural Networks (SNNs) are particularly well-suited for modeling the sparse and time-varying characteristics of sEMG signals[3]. Meanwhile, Hyper-dimensional Computing (HDC) excels at robust pattern recognition by leveraging high-dimensional distributed representations, showing strong performance even with limited training data[4, 5]. Motivated by the complementary strengths of these technologies[6], we propose DSN, a hybrid framework designed for ultra-low-power sEMG signal classification.

The proposed framework consists of two key components. First, the SNN module uses randomized initialized weights with lightweight one-step training to extract discriminative features, which minimizes computational overhead while avoiding complex iterative weight optimization. Second, the HDC module transforms these features into high-dimensional representations that enable robust classification, even in noisy or resource-constrained environments. As is shown in Fig. 1, experimental results demonstrate that our framework achieves energy consumption that is $26.87\times$ lower than that of LSTM and $42.42\times$ lower than that of GRU, while maintaining over 90% accuracy with only 20% of the training data. As the circle's size in the figure increases, so does the quantity of model parameters. Larger circles signify more model parameters, highlighting the DSN model's small parameter count. It also achieves an excellent balance between classification accuracy and energy efficiency, making it particularly suitable for edge deployment in remote healthcare applications.

The main contributions of this work are as follows:

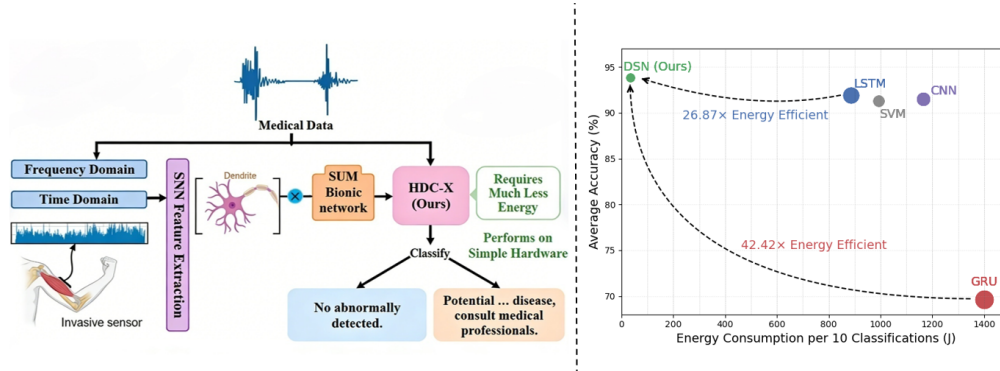


Fig. 1. Energy Consumption vs. Average Accuracy for Different Models.

1. We develop a highly competitive three-class fatigue classifier through the synergistic integration of SNN and HDC. With an average accuracy of 94.40%, the classifier meets the demands of AI-driven remote healthcare applications.
2. The DSN Framework features ultra-low energy consumption enabled by randomized weight design and lightweight one-step training of its SNN component, which strongly supports long-term continuous monitoring on wearable devices.
3. The framework demonstrates strong robustness to environmental noise and excellent adaptability to small-sample scenarios. These characteristics effectively overcome data scarcity challenges commonly encountered in personalized healthcare.

2. Related Works and Background

Muscle fatigue, defined here as localized neuromuscular fatigue, can be effectively assessed by surface electromyography, a noninvasive biosignal that captures neuromuscular activity[7]. sEMG reflects fatigue through spectral shifts toward higher frequencies during muscle exertion, offering real-time monitoring advantages over invasive biochemical or imaging methods. While alternatives like mechanomyography exist, sEMG remains the most established modality for fatigue characterization due to its reliability and minimal signal degradation[8, 9].

2.1. sEMG-based Signal Classification Methods

When local muscle fatigue occurs, significant changes can be observed in both the time-domain and frequency-domain characteristics of surface electromyography signals. These characteristic changes in sEMG signals have become the focus of many research efforts for EMG Signal Classification. However, most existing classification methods rely on single neural networks, which often suffer from high computational complexity and insufficient robustness.

J. Wang et al. [10] collected sEMG signals from participants during cycling and used an LSTM network for sEMG Signal classification. Although LSTM excels in capturing temporal dependencies, its gating mechanisms and long-term memory functionalities require more parameters and computational resources, leading to increased computational overhead. X. Wang et al.[11] approached the problem differently by dividing processed sEMG signals into seven segments, extracting features, and then applying various algorithms, including K-Nearest Neighbors, Logistic Regression, and Support Vector Machine (SVM) for three-level fatigue classification. While SVM achieved the best classification results among these approaches, it requires significant space for storing training samples and kernel matrices. Moreover, SVM’s computational complexity increases when handling high-dimensional features, resulting in limited robustness. Similarly, Q. Liu et al. [12] proposed an improved Whale Optimization Algorithm combined with SVM to enhance the accuracy of dynamic fatigue classification. Despite this optimization, SVM still faces fundamental challenges in computational efficiency and robustness when processing high-dimensional data.

Beyond traditional machine learning, recent years have seen a rapid expansion of deep learning methods for sEMG and biosignal classification[13]. Several comprehensive studies have summarized the shift toward end-to-end feature learning and highlighted key challenges such as cross-subject variability, session inconsistency, and real-time deployment constraints[14]. In particular, one-dimensional convolutional neural networks and residual CNN architectures have demonstrated strong performance on raw EMG signals while maintaining low computational cost[15]. Hybrid CNN–LSTM models have been widely adopted to jointly capture spatial and temporal characteristics in EMG, leading to improvements in gesture recognition and muscle fatigue analysis[16]. More recently, transformer-based architectures and transfer-learning strategies have been introduced for EMG, showing promising generalization across subjects and the ability to adapt with very lim-

ited calibration data[17, 18]. Lightweight multi-stream deep models have also been proposed to reduce inference cost in practical EMG systems[19]. Together, these developments represent an essential and rapidly growing direction in EMG classification research and should be included to provide a complete and up-to-date overview of existing methods.

2.2. Spiking Neural Networks

SNNs represent a class of computational models that emulate the communication mechanisms observed in biological neural systems, distinguishing themselves fundamentally from traditional artificial neural networks[20]. Unlike conventional neural networks that utilize continuous values for information representation and transmission, SNNs employ discrete spikes or pulses at precise temporal intervals as information carriers, thereby more accurately mimicking the brain’s information processing mechanisms[21]. This distinctive operational paradigm positions SNNs as particularly advantageous for applications demanding low energy consumption and rapid response times.

To enhance the biological plausibility of our neural system simulation, we implement the Leaky Integrate and Fire (LIF) model as our foundational architecture[22]. The LIF model offers a computationally efficient yet biologically relevant mathematical framework for modeling neuronal behavior. Within this model, each neuron operates analogously to a leaky capacitor: incoming stimuli from connected neurons or external sources accumulate until reaching a predetermined threshold, triggering a spike generation event. Subsequently, the neuron enters a refractory period characterized by temporary unresponsiveness to further stimuli. This biologically-inspired mechanism not only captures the essential characteristics of neuronal operation but also confers exceptional temporal processing capabilities to networks constructed using this model, rendering them particularly effective for real-time processing applications.

For a discrete-time Spiking Neural Network (SNN) with n neurons over T time steps, its dynamics are governed by:

$$V_{t+1} = AV_t + Bx_t - Rs_t, \quad s_t = H(V_t - \theta) \in \{0, 1\}^n, \quad (1)$$

where A, B, R are network parameters. The linear readout layer is expressed as:

$$y = C \sum_{t=1}^T \alpha^{T-t} \phi(V_t), \quad 0 < \alpha \leq 1. \quad (2)$$

Through scaling operations using inequalities, the Lipschitz constant can be derived as follows:

$$\begin{aligned}
\|\Delta y\|_2 &\leq \|C\|_2 \sum_{t=1}^T \alpha^{T-t} \|\Delta V_t\|_2 \\
&= \|C\|_2 \|B\|_2 \sum_{t=1}^T \alpha^{T-t} \sum_{k=1}^t \|A\|_2^{t-k} \|\Delta x_k\|_2.
\end{aligned} \tag{3}$$

Based on the above derivation, we obtain Theorem 1 for SNNs, which lays the foundation for the analysis of DSN analysis:

Theorem 1 (piecewise-Lipschitz). *The mapping $\mathcal{F} : X \mapsto y$ of SNN is piecewise-affine. For X, X' in the same region, we have $\|\Delta y\|_2 \leq L_S \|X - X'\|_2$.*

2.3. Hyperdimensional Computing

Hyperdimensional Computing represents a brain-inspired computational paradigm that fundamentally differs from traditional numerical approaches. HDC employs high-dimensional vectors, typically ranging from hundreds of thousands to millions of dimensions, for data representation, offering distinct advantages in complex pattern recognition and memory-related tasks[5].

Conventional numerical computing systems, which typically utilize scalars or low-dimensional vectors, encounter significant challenges when processing high-dimensional data, primarily due to the "curse of dimensionality" phenomenon. As dimensionality increases, these systems suffer from rapidly escalating data sparsity and computational complexity[23]. HDC addresses these limitations through its unique high-dimensional space representation, where each data element is encoded as a hypervector, a high-dimensional binary vector with significantly greater dimensionality than traditional representations.

At the core of HDC lies the formation of class-specific hypervectors, which are generated by systematically combining hypervectors belonging to the same class. To ensure robust classification performance, the framework incorporates an iterative self-checking mechanism that continuously refines class assignments. This mechanism identifies and corrects misclassifications by removing erroneous vectors from their current class hypervectors and reassigning them to their appropriate classes. The classification process is further

strengthened through precise verification using Hamming distance calculations, which enable accurate identification of the most similar hypervector for each data point.

3. DSN Framework for sEMG Signal Classification

The proposed DSN architecture for sEMG-based fatigue classification comprises four core modules, as illustrated in Fig.2. These modules include: a preprocessing module that extracts multi-domain features from raw signals, generating an N-dimensional feature vector per sample; SNN feature extraction, where a spiking neural layer with leaky integrate-and-fire neurons processes these features through temporal dynamics; Training, which encodes spike patterns into D-dimensional hypervectors via binding operations, followed by Hamming distance-based classification against pre-trained prototypes; and Testing, which evaluates classification robustness under signal variability.

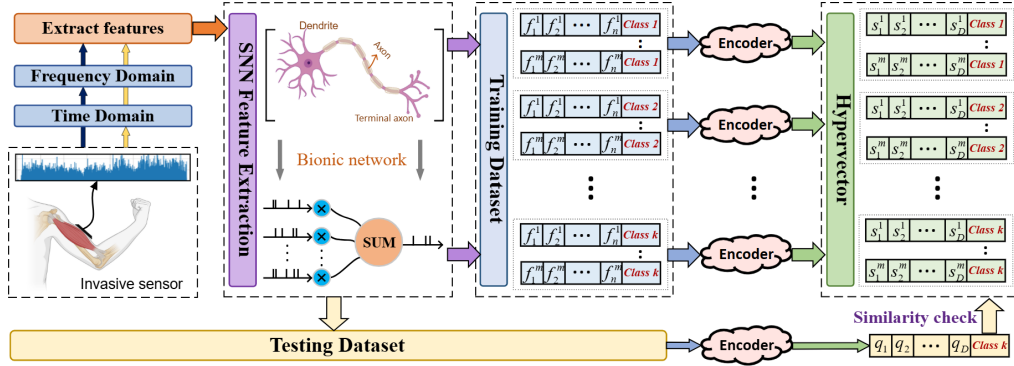


Fig. 2. Framework workflows. The proposed framework integrates spiking neural networks for spatiotemporal feature extraction and hyperdimensional computing for efficient classification. SNN captures dynamic EMG patterns through bio-inspired spike encoding, while HDC maps features into a high-dimensional space for classification.

3.1. Preprocessing

Effective classification requires careful signal preprocessing and discriminative feature extraction. We implement a 20-350Hz band-pass filter to enhance signal fidelity while preserving fatigue-sensitive components [24].

To capture the nonstationary nature of sEMG signals, we employ complementary time-domain and frequency-domain analyses, extracting eight

clinically interpretable features (see Table 1 for details). Time-domain features quantify signal intensity and approximate frequency content, while frequency-domain features track spectral dynamics, neuromuscular adaptations, and time-frequency characteristics [25, 26].

Notably, integrated sEMG (IEMG) and mean absolute value (MAV) reflect increasing muscle activation intensity, while mean frequency (MNF) and median frequency (MDF) capture progressive spectral compression during fatigue. Their complementary trends validate the multi-domain approach’s ability to characterize fatigue dynamics, providing discriminative inputs for subsequent SNN-based feature extraction and HDC classification. This robust feature set addresses the inherent non-stationarity of sEMG signals, offering a significant advantage over single-domain approaches [27].

Table 1: Preprocessing & Features

Category	Items	Description
Preprocessing	20-350Hz band-pass	Enhance fidelity, preserve fatigue
Time-domain	IEMG	Signal intensity (total energy)
	MAV	Signal intensity (avg amplitude)
	ZCR	Approximate frequency content
Frequency-domain	MNF	Spectral shifts (fatigue)
	MDF	Spectral shifts (fatigue)
	PF	Neuromuscular adaptations
	FR	Neuromuscular adaptations
	WE	Time-frequency info (nonstationarity)

3.2. Extract Features

In this work, the SNN serves as the feature extractor to mine spatio-temporal dynamic patterns from raw sEMG signals. Its design combines the bionic characteristics of biological neurons with engineering optimization strategies.

After the data are transferred into the memory, they will be sent to the SNN for processing. We employ a LIF neuron model as the core computational unit of the SNN. The membrane potential $V(t)$ of a LIF neuron evolves according to the following differential equation:

$$\tau \frac{dV(t)}{dt} = -V(t) + I(t) \quad (4)$$

where τ is the membrane time constant, and $I(t)$ represents the synaptic input current. When $V(t)$ exceeds a predefined threshold, the neuron emits a spike, and the membrane potential is reset to a baseline value of V_{rest} . After discretization, the membrane potential update rule becomes:

$$V[t] = V[t - 1] + \frac{1}{\tau} (-V[t - 1] + \mathbf{W} \cdot \mathbf{x}[t]) \quad (5)$$

In this equation, τ controls the potential decay rate, $\mathbf{W} \in \mathbb{R}^{N \times M}$ is the weight matrix of the fully connected layer, and the input $\mathbf{x}[t]$ is the sEMG signal after standardization and row-level normalization. When the membrane potential exceeds the threshold V_{th} , the neuron triggers a pulse and resets the potential to the resting state. This mechanism simulates the action potential discharge process of biological neurons and converts continuous signals into sparse pulse trains.

To capture the transient muscle activation pattern of the sEMG signal, the input data is repeatedly fed into the SNN at T time steps. The pulse output $\mathbf{s}[t] \in \{0, 1\}^M$ of each time step is accumulated and the average firing rate is calculated:

$$\mathbf{F} = \frac{1}{T} \sum_{t=1}^T \mathbf{s}[t] \quad (6)$$

Each element of the feature vector $\mathbf{F} \in \mathbb{R}^M$ represents the activation frequency of the corresponding neuron in the time window, compressing dynamic pulse activity into a static statistic. This design retains the timing characteristics of muscle contraction (such as burst activation and relaxation cycles) while providing dimensionally regular input for subsequent classifiers.

Notably, the SNN's weight matrix is initialized with a Gaussian distribution to ensure diverse initial responses without manual tuning. Unlike traditional deep learning models that require iterative weight optimization, our SNN only performs a lightweight one-step training using surrogate gradient, which balances feature discriminability and computational efficiency, aligning with the energy-saving goal for edge deployment.

To overcome the non-differentiability of the impulse function, SNN uses inverse tangent instead of the gradient to achieve end-to-end training:

$$\sigma'(x) = \frac{1}{\pi(1 + (\pi x)^2)} \quad (7)$$

This function provides a smooth gradient approximation near the pulse trigger point ($x=0$), enabling weight matrix update via a lightweight one-step

back-propagation, avoiding the heavy computational overhead of iterative training while enhancing feature discriminability. The weights are initialized using a Gaussian distribution with a mean of 0 and a standard deviation of 1 to ensure diversity of initial response and stability of training.

3.3. Encoding

The encoding module maps the spatiotemporal features extracted by the hidden layer of the SNN to a high-dimensional binary space suitable for HDC. Let the feature vector output from the SNN hidden layer be $f \in \mathbb{R}^H$, where H is the dimension of the hidden layer, and perform a linear transformation using a random projection matrix $W \in \{-1, +1\}^{H \times D}$, D is the dimension of the hyperdimensional space:

$$hV = f \cdot \mathbf{W} \quad (8)$$

where $h' \in \mathbb{R}^D$. To enhance robustness against noise and enable energy-efficient bitwise operations, we binarize the projected vector using the sign function. The random projection matrix $\mathbf{W}$ is fixed during training, eliminating backpropagation overhead while ensuring approximate orthogonality between different feature mappings – a key property that enables HDC’s error resilience. Through this encoding scheme, the temporal dynamics captured by SNN spikes are embedded into stable geometric relationships within the hyperdimensional space, allowing subsequent HDC classifiers to perform efficient similarity-based reasoning through hardware-friendly XOR operations.

In encoding process, multiple calculations are considered. These notations, definitions, lemmas and propositions that lead to the following theorems are seen in Appendix.

3.4. Training with HDC

The training process implements the classification task of sEMG signals through HDC. The core idea is to combine symbolic operations in high-dimensional space with biomimetic feature extraction of SNN to construct an efficient and robust classifier. The training process is divided into two main stages: hyperdimensional generation and hyperdimensional classification.

3.4.1. Stage One(Hyperdimensional generation):

During the training process, the feature vector $f \in \mathbb{R}^H$ extracted by SNN is mapped to a D -dimensional hyperdimensional space through a random

projection matrix W . The projection result is binarized using a sign function to generate a hyperdimensional vector $H \in \{-1, +1\}^D$: $H = \text{sign}(h)$.

The binarization operation compresses storage space and enhances the model's noise resistance, making it suitable for deployment on resource-constrained edge devices. Given the significant individual differences and noise interference in sEMG signals, hyperdimensional encoding constructs feature representations that are insensitive to distribution shifts through the randomness and symbolic operations of high-dimensional space.

To precisely characterize its noise robustness, we first consider two feature vectors $x^{(1)}$ and $x^{(2)}$ satisfying $\frac{|x_n^{(1)} - x_n^{(2)}|}{\Delta_n} \leq \delta$ for all features $n \in \{1, \dots, d\}$, where Δ_n denotes the range of the n -th feature and $\delta \in [0, 1]$. For the n -th feature, let $L^{(l_n(x_n^{(1)}))}$ and $L^{(l_n(x_n^{(2)}))}$ be the corresponding level hypervectors; their Hamming distance is bounded by:

$$d_H(L^{(l_n(x_n^{(1)}))}, L^{(l_n(x_n^{(2)}))}) \leq \delta_d := \frac{M}{M-1}\delta + \frac{2}{M-1}. \quad (9)$$

After multiplying with the respective feature identifiers, the hypervectors

$$T^{(n,1)} = L^{(l_n(x_n^{(1)}))} \otimes ID^{(n)}, \quad T^{(n,2)} = L^{(l_n(x_n^{(2)}))} \otimes ID^{(n)} \quad (10)$$

satisfy:

$$d_H(T^{(n,1)}, T^{(n,2)}) \leq \delta_d. \quad (11)$$

For sufficiently large D , the Hamming distance between $T^{(n,1)}$ and $T^{(n,2)}$ can be modeled as a Bernoulli process across dimensions:

$$\mathbb{I}(T_j^{(n,1)} \neq T_j^{(n,2)}) \sim \text{i.i.d. Bernoulli}(\delta_d), \quad j = 1, \dots, D. \quad (12)$$

The final hypervectors are obtained by bundling all feature hypervectors:

$$S^{(1)} = \bigoplus_{n=1}^d T^{(n,1)}, \quad S^{(2)} = \bigoplus_{n=1}^d T^{(n,2)}. \quad (13)$$

For the j -th index, define $Z_j = \mathbb{I}(S_j^{(1)} \neq S_j^{(2)})$ and let $p_j^{(1)}$ denote the number of +1 entries in $\{T_j^{(n,1)}\}_{n=1}^d$. Using Bayesian reasoning, we derive:

$$P(Z_j = 1) = \frac{C_d^n}{2^d}. \quad (14)$$

Consider the conditional distribution $(p_j^{(2)} \mid p_j^{(1)})$, we obtain:

$$\begin{aligned} P(Z_j = 1 \mid p_j^{(1)} = n) &= \frac{P(Z_j = 1, p_j^{(1)} = n)}{P(Z_j = 0, p_j^{(1)} = n) + P(Z_j = 1, p_j^{(1)} = n)} \\ &= \sum_{k=\lceil d/2 \rceil}^d \sum_{i=\max(0, k-d+n)}^{\min(n, k)} C_n^i C_{d-n}^{k-i} \delta_d^{n+k-2i} (1 - \delta_d)^{d-n-k+2i}. \end{aligned} \quad (15)$$

By the law of large numbers, the average over all indices converges to the expectation as $D \rightarrow \infty$:

$$\frac{1}{D} \sum_{j=1}^D Z_j \xrightarrow{D \rightarrow \infty} \mathbb{E}[Z_j] = g(\delta_d). \quad (16)$$

Hence, the Hamming distance between $S^{(1)}$ and $S^{(2)}$ is almost surely bounded by $g(\delta)$, confirming robustness to input noise. This leads to Theorem 2:

Theorem 2 (Robust to Input Noise). *Let $s^{(1)}, s^{(2)} \in \mathcal{S} \subseteq \mathbb{R}^d$ be two feature vectors. Let $S = f_{\mathcal{ID}, \mathcal{L}, \Theta}(s)$ denote the hypervector mapping. Suppose that for all features $n \in \{1, \dots, d\}$,*

$$\frac{|x_n^{(1)} - x_n^{(2)}|}{\Delta_n} \leq \delta, \quad (17)$$

where Δ_n is the range of the n -th feature and $\delta \in [0, 1]$. Then, with sufficiently large D , the expected Hamming distance between $S^{(1)}$ and $S^{(2)}$ converges to a monotonically increasing function $g(\delta)$:

$$\mathbb{E}[d_H(S^{(1)}, S^{(2)})] \rightarrow g(\delta). \quad (18)$$

Through Theorem 2, we exhibit that this encoding inherently resists input noise. Specifically, when noise corrupts the feature vector $s^{(1)}$, which yields a perturbed version $s^{(2)}$, the theorem establishes that the Hamming distance between their respective Sample-HVs, $S^{(1)}$ and $S^{(2)}$, is bounded above. This bound remains valid provided the noise magnitude is constrained by a relative ratio δ . Notably, the upper limit, denoted $g(\delta)$, stays numerically small even under moderate noise conditions.

3.4.2. Stage Two(Hyperdimensional classification):

During the training process, the system accumulates the same class of hyperdimensional vectors into the corresponding class prototypes based on the sample labels:

$$C_k = \sum_{i \in \text{Class } k} H_i. \quad (19)$$

Among them $C_k \in \mathbb{Z}^D$ is the prototype vector of class k . The accumulation mechanism essentially constructs class centroids in high-dimensional space, enhancing the common features of similar samples and amplifying the differences between different ones. During the testing phase, the input sample's hyperdimensional vector f_n is classified by matching it with each class prototype C_k using Hamming distance:

$$\hat{y} = \arg \max_k f_n \cdot C_k^T. \quad (20)$$

The dot product operation calculates the similarity between two vectors, where a larger value indicates a higher degree of matching between the sample and the category. Hamming distance matching is achieved through bitwise operations, providing high computational efficiency suitable for real-time classification tasks. The symbolic operation of HDC and Hamming distance matching ensure classification accuracy while significantly reducing computational complexity, meeting the real-time requirements of wearable devices.

If we define $S^{(1)}, S^{(2)}, \dots, S^{(N)} \in \mathcal{H}^D$ as independently sampled random hypervectors, define their sum as $C = \left[\sum_{n=1}^N S^{(n)} \right]$, define $Z_i^{(j)} = \sum_{k \neq j} S_i^{(k)}$ and $C_i = \left[S_i^{(j)} + Z_i^{(j)} \right]$, we have:

$$d_H(C, S^{(j)}) \sim \frac{1}{D} \text{Binomial}(D, \frac{1}{2} - p(N)), \quad (21)$$

$$d_H(C, S^*) \sim \frac{1}{D} \text{Binomial}(D, \frac{1}{2}). \quad (22)$$

where S^* denotes an arbitrary random hypervector. Applying the Central Limit Theorem as $D \rightarrow \infty$, these distributions converge to:

$$d_H(C, S^{(j)}) \sim \mathcal{N}\left(\frac{1}{2} - p(N), \left(\frac{1}{4} - p(N)^2\right)D^{-1}\right), \quad (23)$$

$$d_H(C, S^*) \sim \mathcal{N}\left(\frac{1}{2}, \frac{1}{4}D^{-1}\right). \quad (24)$$

The probability of a random hypervector being closer than a constituent hypervector satisfies

$$P(d_H(C, S^{(j)}) < d_H(C, S^*)) \rightarrow 1 \quad \text{as } D \rightarrow \infty, \quad (25)$$

since the means of the two distributions are separated by $p(N)$ and variances vanish as D^{-1} .

This implies that any constituent hypervector is almost surely closer to the cluster prototype C than a randomly sampled hypervector, leading to Theorem 3:

Theorem 3 (Distance Between Cluster Prototype and Constituents).

Let $S^{(1)}, S^{(2)}, \dots, S^{(N)} \in \mathcal{H}^D$ be independently sampled random hypervectors.

Define their sum as $C = \left[\sum_{n=1}^N S^{(n)} \right]$.

As $D \rightarrow \infty$, for any random hypervector $S^ \in \mathcal{H}^D$ and any index $j \in \{1, \dots, N\}$, the Hamming distance satisfies*

$$P(d_H(C, S^{(j)}) < d_H(C, S^*)) \rightarrow 1. \quad (26)$$

Theorem 3 offers theoretical justification for our bundling-based clustering approach. It establishes that the Hamming distance between the Cluster-HV C and any constituent Sample-HV of the cluster is, in most cases, smaller than the characteristic value $d_H = 0.5$ observed between two arbitrary hypervectors. This finding validates the Cluster-HV's capacity to adequately encapsulate all cluster members. Furthermore, given that Sample-HVs within the same cluster generally exhibit greater mutual similarity than randomly selected hypervectors, the Cluster-HV is anticipated to capture even more robust interrelationships in practical scenarios than what the theorem formally guarantees under the random hypervector premise.

3.5. Theoretical Proof of Availability

To assess the resilience of DSN in this regard, we performed a perturbation analysis where elements of all stored Cluster-HVs were randomly inverted (switching +1 to -1 and vice versa) to mimic hardware-related instabilities. We can consider the i -th index. Define indicator variables:

$$X_i = \mathbf{1}_{\{(C'_1)_i \neq S_i\}}, \quad Y_i = \mathbf{1}_{\{(C'_2)_i \neq S_i\}}. \quad (27)$$

The post-corruption Hamming distances are

$$d_H(S, C'_1) = \frac{1}{D} \sum_{i=1}^D X_i, \quad d_H(S, C'_2) = \frac{1}{D} \sum_{i=1}^D Y_i. \quad (28)$$

We can calculate that

$$\mathbb{E}[Y_i] - \mathbb{E}[X_i] = \epsilon(1 - 2p) > 0. \quad (29)$$

Define

$$Z = \sum_{i=1}^D (Y_i - X_i), \quad (30)$$

so that

$$\mathbb{E}[Z] = D \cdot \epsilon(1 - 2p). \quad (31)$$

Since X_i, Y_i are independent and bounded, Hoeffding's inequality implies

$$\mathbb{P}(Z < 0) \leq \exp(-cD) \quad (32)$$

for some constant $c > 0$. Equivalently,

$$\mathbb{P}(d_H(S, C'_1) \leq d_H(S, C'_2)) = \mathbb{P}(Z > 0) \rightarrow 1 \quad \text{as } D \rightarrow \infty. \quad (33)$$

This shows that even with random bit flips, the relative ordering of Hamming distances is preserved with high probability.

Theorem 4 shows that, when the dimensionality is sufficiently large, flipping up to half of the elements results in only negligible effects on classification performance.

Theorem 4 (Robustness to Hardware Error). *Let S be a sample hypervector, and C_1, C_2 be two cluster hypervectors with initial Hamming distances*

$$d_H(S, C_1) = d_1, \quad d_H(S, C_2) = d_2, \quad d_2 - d_1 = \epsilon > 0. \quad (34)$$

If a fraction $p < 0.5$ of bits in both C_1 and C_2 are randomly flipped, producing C'_1 and C'_2 , then as $D \rightarrow \infty$:

$$P(d_H(S, C'_1) < d_H(S, C'_2)) \rightarrow 1. \quad (35)$$

4. Experiment and Results

4.1. Experimental Setup

1) *Datasets*: All experiments in this study are conducted based on the public Cerqueira’s dataset [28], while the self-collected real-world dataset is only used for additional performance verification.

Cerqueira’s dataset contains raw surface electromyography data collected using the Delsys Trigno system. This dataset focuses on eight muscles (four in each arm) of 13 healthy adult participants, with a sampling frequency of 1259 Hz. It supports research on muscle fatigue, multi - muscle coordination, and activity patterns under different fatigue states.

The real-world dataset was meticulously gathered in a strictly controlled environment, ensuring high-quality and reliable data. This dataset includes six surface electromyography signals with a sampling frequency of 2148.1481 Hz, divided into three classes representing relaxation, mild fatigue, and fatigue states. These data were collected using the BiTalino device, whose high sampling frequency and precision enable capturing subtle changes in muscle activity. It will be publicly released to facilitate muscle fatigue research.

Muscle fatigue is influenced by physiological variables such as age, gender, muscle mass, and physical activity level [29, 30]. To ensure robust and generalizable findings, we selected 18 healthy adult participants, all screened for underlying medical conditions, musculoskeletal disorders, or recent injuries that could affect muscle performance. To reduce impedance, the biceps brachii area was cleaned, shaved if necessary, and abraded with sandpaper. Surface electrodes were placed 2–3 cm apart along the muscle belly and secured with medical tape. Participants first determined their maximum biceps curl weight, with 40% of this weight used as the test load. sEMG signals were recorded for 30–60 seconds with muscles relaxed. Participants then performed isometric or isotonic contractions (e.g., holding a 90-degree arm bend or slow elbow flexion), with sEMG signals recorded during contractions. Each contraction was repeated 3–5 times, with rest intervals to prevent excessive fatigue.

2) *Implementation*: Python was used to filter, denoise, and segment sEMG signals: first 10s (relaxation), 35–45s (mild fatigue), last 10s (fatigue). Eight time- and frequency-domain features were extracted from these three classes. Our project was implemented in PyTorch. All experiments were run on an i5-12500H processor (2.50 GHz) with 16GB of memory.

The SNN module for feature extraction adopts a single hidden layer architecture, aligned with the lightweight design goal for edge deployment. Detailed parameters are specified as follows:

1. **Input layer:** 8 neurons, corresponding to the 8 multi-domain features (3 time-domain: IEMG, MAV, ZC; 5 frequency-domain: MNF, MDF, PKF, FR, WENT) extracted from preprocessed sEMG signals.
2. **Hidden layer:** 128 LIF neurons. Key hyperparameters include a membrane time constant (τ) of 2.0, a firing threshold (V_{th}) of 1.0, and a resting potential (V_{rest}) of 0.0. The weight matrix ($W \in \mathbb{R}^{8 \times 128}$) is initialized using a Gaussian distribution, whose mean=0, while standard deviation=1, and remains untrained to avoid complex optimization, consistent with our core design of eliminating backpropagation.
3. **Time steps:** Input features are iteratively fed into the SNN over 10 time steps ($T = 10$) to capture transient muscle activation dynamics. The pulse output of each time step is accumulated to calculate the average firing rate as the final feature representation.

4.2. Comparison with Previous Work

We evaluated the performance of our proposed DSN against several well-established architectures: 1D-CNN [31], LSTM [10], SVM [32] and GRU[33], all widely used for sEMG signal classification. All comparative models' accuracies are independently reproduced following the original literatures, with consistent data preprocessing and 10-fold cross-validation to ensure fair comparison. Energy consumption is measured via NVIDIA Nsight Systems, which captures only core computations, and the results are averaged over five repeated tests for stability.

The experimental results for various machine learning and deep learning models applied to EMG fatigue classification are presented in Table 2. The 10-Folds Accuracy for each model was recorded. It was observed that the Deep Supervised Network (DSN) consistently achieved the highest classification accuracy among all evaluated architectures. Other models, including Support Vector Machine (SVM), Convolutional Neural Network (CNN), and Long Short-Term Memory (LSTM), exhibited accuracies that were generally comparable to each other but lower than that of the DSN. The Gated Recurrent Unit (GRU) model, in contrast, showed a noticeably lower classification accuracy compared to all other models.

Energy consumption during both the training and testing phases was also measured for each model, as detailed in Table 2. The DSN model exhibited

significantly lower energy consumption during both its training and testing operations compared to the other models. Specifically, its testing energy consumption was found to be notably reduced. For the SVM, CNN, and LSTM models, training energy values were relatively low, typically in the order of 10^3 Joules. However, their corresponding testing energy requirements were considerably higher. Modern methods proposed in recent years, at the same time, also have disadvantages in energy consumption and accuracy compared with DSN method.

Overall, our DSN outperformed both the CNN and LSTM in terms of accuracy and efficiency, particularly in the three-class classification task, while also being more computationally efficient and hardware-friendly.

Table 2: Performance and Energy Consumption of Different Models for EMG Fatigue Classification

Models	Accuracy (%)	Energy (train J)	Energy (test J)	Memory (MB)	Pre-inference Latency (ms)
SVM[32]	91.26 ± 0.04	1281 ± 29.55	993.6 ± 40.8	133.24	0.0572
1D-CNN[31]	91.49 ± 0.02	2131 ± 55.46	1165.9 ± 31.4	1036.46	0.1928
LSTM[10]	91.92 ± 0.04	2491 ± 47.22	886.9 ± 34.6	1085.45	0.2774
GRU[33]	69.61 ± 0.03	5221 ± 56.46	1399.9 ± 45.4	1102.40	1.0607
TCN[34]	94.00 ± 3.77	5.45 ± 0.37	63.35 ± 33.99	27.65	0.2937
ViT[35]	93.80 ± 3.82	11.30 ± 1.73	45.07 ± 9.45	27.37	0.4349
SNN[36]	87.38 ± 2.98	3.99 ± 1.92	43.82 ± 13.15	967.94	0.3046
DSN	94.40 ± 0.02	3.31 ± 0.28	34.09 ± 11.98	951.76	0.1488

4.3. Confusion Matrix Analysis

As illustrated in Figure 3, the confusion matrices for the DSN, CNN, LSTM, and SVM models on the test dataset are presented. In these matrices, the values along the main diagonal represent the proportion of correctly classified samples for each respective state, while off-diagonal entries indicate misclassifications. For the purpose of this study, the labels 0, 1, and 2 on the axes represent the relax, little fatigue, and fatigue, respectively, reflecting the progressive nature of muscle fatigue. GRU is not included here as it achieves only 69.61% classification accuracy, significantly lower than other baseline models, with widespread random misclassifications that provide no meaningful discriminative insights for comparative analysis.

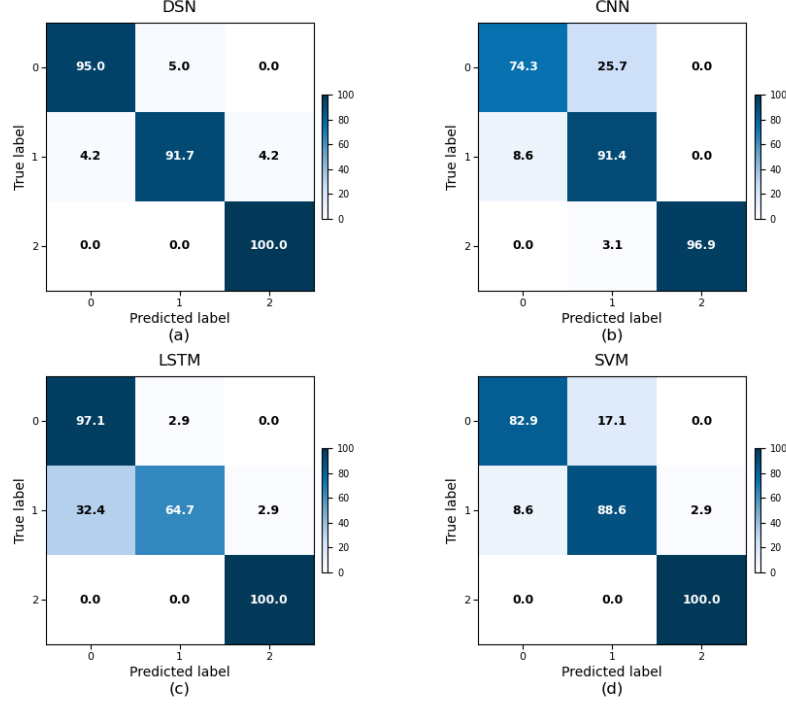


Fig. 3. Confusion matrix for 4 models

Upon visual inspection of the confusion matrices, it is consistently observed that most models generally exhibit relatively higher recognition accuracy for the Class 0 and Class 2. This can be attributed to the more distinct physiological characteristics or signal features that define the extreme ends of the fatigue spectrum, making them more separable in the feature space. Notably, all models achieved high or perfect classification rates for the little fatigue.

However, classification performance for the Class 1 varied considerably among models. The DSN model excelled in classifying this state, achieving high accuracy with minimal confusion. Conversely, LSTM exhibited notably weaker performance for this critical transitional state, frequently misclassifying Class 1 samples as Class 0, suggesting limitations in effectively discerning its subtle boundaries. Similarly, CNN and SVM models also faced challenges, primarily showing confusion between Class 0 and Class 1 states, highlighting

inherent ambiguities in the feature space for these architectures.

4.4. Feature Ablation Study

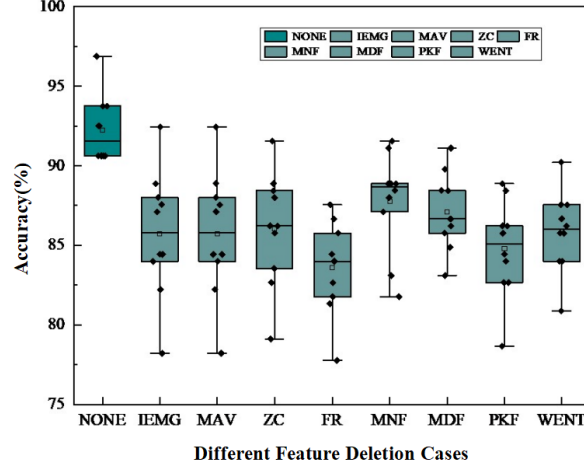


Fig. 4. Accuracy distribution for different feature deletion cases.

To comprehensively ascertain the individual contribution of each extracted sEMG feature, a detailed feature ablation study was conducted. This involved systematically evaluating the model’s accuracy when one specific feature was removed from the complete set of eight selected features. Figure 4 visually represents the distribution of accuracies for each deletion case.

Removing any single feature noticeably decreased classification accuracy, confirming the discriminative power of all selected features. Particularly, ablating Frequency Ratio resulted in the most significant performance degradation.

This analysis validates our multi-domain feature extraction approach, reinforcing that all eight selected time-domain and frequency-domain features contribute synergistically to robust and accurate EMG fatigue classification. The findings underscore the necessity of a rich and diverse feature set to effectively capture the complex and non-stationary characteristics inherent in sEMG signals across different fatigue states.

4.5. Impact of Dimensionality

The dimensionality of HDC determines the correlation between hypervectors, which may directly affect the recognition accuracy. The dimensionality

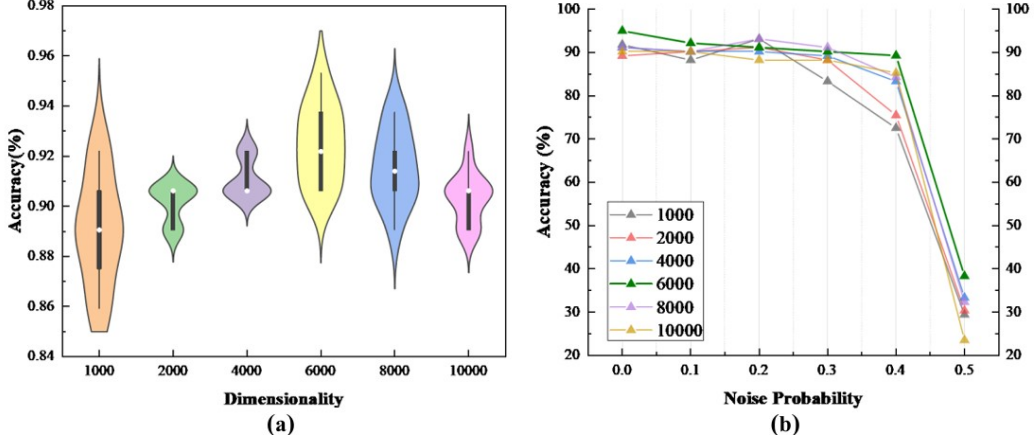


Fig. 5. (a) Accuracy of the model at varying hypervector dimensions. (b) Robustness of the model under increasing levels of bit-flip noise. Higher-dimensional hypervectors maintaining high accuracy even at significant noise levels.

is denoted by D , the input feature dimension, and the hidden neurons set to 1000. In the SNN, the time constant of the LIF neurons was set to 2.0, and the number of time steps T was initially set to 10. Fig.5(a) shows the accuracy of our model at different dimensionality levels. It can be seen that our model perfectly inherits the advantages of the HDC algorithm, demonstrating good precision at high dimensionality.

4.6. Anti-Interference Capability Analysis

Robustness analysis ensures that the neural network maintains stable output when faced with minor changes in input data. Typically, if bits in floating-point numbers are flipped, the resulting numerical errors can be exponential.

In this experiment, we systematically varied the hypervector dimension across multiple values D and introduced different noise levels to the hypervectors. We trained the model for each dimension and evaluated its accuracy under increasing noise levels. As illustrated in Fig.5(b), the results revealed a clear trend: higher-dimensional hypervectors exhibit significantly greater robustness to noise.

For instance, when $D = 6000$, the model maintained an accuracy of 90.12% even with 40% of the elements flipped, representing only a 5.18% decrease from the noiseless baseline. In contrast, lower-dimensional hypervectors, for instance, $D = 1000$ showed a more pronounced decline in accuracy

as noise increased, dropping to 72.55% at 40% noise. This demonstrates that our neural network is highly robust and has a high tolerance for hardware errors.

4.7. Fault Tolerance Evaluation

4.7.1. Accuracy Comparison of DSN and SNN

Bit Error Rate (BER) is a critical metric that quantifies the reliability of data transmission or storage systems by measuring the ratio of erroneous bits to the total number of bits processed. To investigate the impact of BER on various neural network architectures, we introduced simulated bit errors into both data and model parameters to accurately replicate challenging real-world scenarios where BER constitutes a significant operational factor. This approach allowed us to create controlled testing conditions while maintaining experimental validity.

The comparative analysis focused specifically on two distinct neural architectures: our proposed DSN framework and conventional SNN models integrated with alternative classification methodologies. For the latter, we selected two widely recognized classifiers, which are Multilayer Perceptrons (MLP) and Variational Autoencoders (VAE), to ensure performance evaluation across different computational paradigms.

We conducted performance assessments across a graduated range of BER levels, beginning with severe error conditions (0.8) and progressing to near-optimal scenarios (0.0001). The results are shown in Fig.6. Our approach maintain higher accuracy at all BER levels, especially under low to medium BER conditions, while SNN combined with MLP and SNN combined with VAE show greater sensitivity to bit errors, especially at higher BER levels.

Fig.7(a) and Fig.7(b) illustrate the accuracy of using SNNs with and without random weights in independent configuration and combination with HDC. The results indicate that the single-layer SNN with random weights achieved the highest accuracy among all test configurations. This discovery highlights the efficiency of random weights in simplifying architectures while maintaining competitive performance.

Traditional methods typically rely on multi-layer architectures and carefully adjusted weights, which are computationally expensive and time-consuming to train. By introducing random weights, the randomness may serve as a form of regularization to prevent overfitting and enhance generalization ability, demonstrating the potential of hybrid models to leverage the advantages

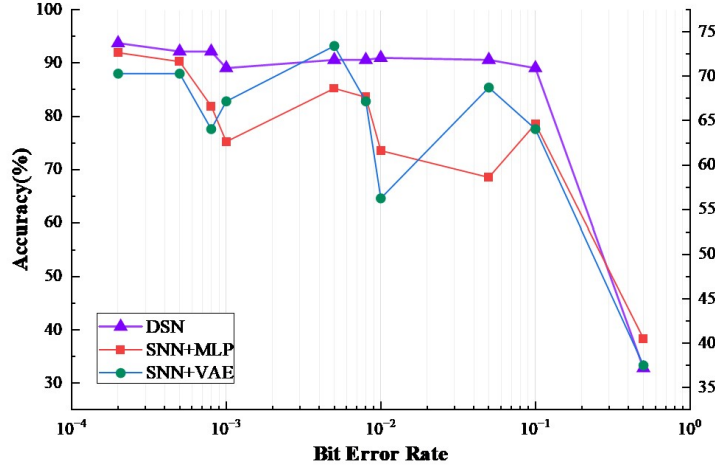


Fig. 6. Comparison of the accuracy of DSN and SNN with other classifiers under bit error conditions.

of both methods. We aim to investigate whether simpler and less resource-intensive models can achieve comparable or even better performance. This is particularly important for applications in energy-efficient hardware or real-time systems with limited computing resources.

Consistent with our design philosophy, the single-layer SNN with randomized initialized weights (plus lightweight one-step training) achieves the highest accuracy, verifying that lightweight training does not compromise efficiency while enhancing feature discriminability.

4.7.2. The Critical Contribution of SNN Layers to HDC Performance

To thoroughly investigate the performance landscape of our DSN framework and, critically, to assess the indispensable role of SNN layers in overcoming the limitations of standalone HDC, a comprehensive ablation study was conducted.

Table 3: Results of Ablation Study on SNN Layer Integration

Model Setup	10-Folds Accuracy	Standard Deviation
HDC	82.30%	5.85
HDC + SNN	94.40%	0.02

As presented in Table 3, the standalone HDC system served as the baseline for comparison, demonstrating an initial classification capability. A sig-

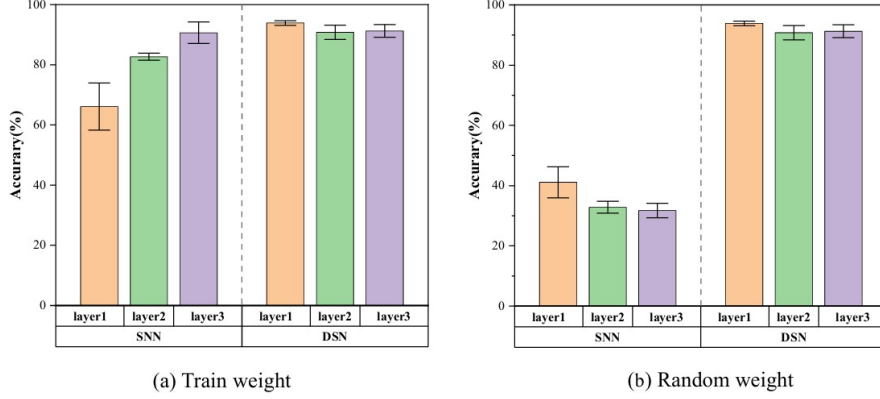


Fig. 7. (a)The impact of data sets at different layers of SNN and trained weights on the training accuracy of SNN and DSN.(b)The impact of data sets at different layers of SNN and random weights on the training accuracy of SNN and DSN.

nificant improvement in accuracy was observed immediately upon the integration of a single SNN layer with the HDC framework. This configuration achieved the highest performance across all tested setups, accompanied by a notably reduced standard deviation. This substantial enhancement underscores the potent synergistic effect between the feature encoding of the SNN and the high-dimensional pattern matching inherent in HDC.

4.8. Sparsity Mitigation via SNN-HDC Synergy

This section provides a detailed explanation of why the proposed DSN framework exhibits strong robustness and few-shot adaptability, thereby effectively mitigating the impact of data sparsity.

1. **Randomized-weight SNN serves as a stable spatiotemporal feature projector.** Its piecewise-Lipschitz property ensures that small input variations—typical in sparse or imperfect datasets—lead to bounded variations in the extracted representations. Because the SNN does not rely on backpropagated weight optimization, it avoids overfitting to scarce samples and produces consistent features even under limited supervision.
2. **HDC module provides intrinsic few-shot learning capability.** High-dimensional binary hypervectors obey concentration-of-measure properties, allowing class prototypes to converge reliably using only a

handful of samples. Theorems 2 and 3 formally guarantee that perturbations, incompletely sampled distributions, or moderate noise only induce bounded Hamming deviations. As a result, the class prototypes remain stable and discriminative even in small-data regimes.

3. **SNN–HDC combination forms a complementary robustness mechanism.** SNN reduces intra-class variance through temporal smoothing, while HDC amplifies class-consistent structure in a high-dimensional symbolic space where noise becomes statistically diluted. This synergy directly compensates for the instability caused by limited training data, explaining why DSN maintains high accuracy even when trained with only 20% of the dataset.

5. Conclusion

This work presented the DSN framework for sEMG signal classification, integrating Spiking Neural Networks (SNNs) with Hyperdimensional Computing (HDC) to exploit their complementary strengths. The SNN component performs efficient, event-driven feature extraction, while the HDC module provides symbolic computation for robust classification. Experiments show that DSN achieves a peak test accuracy of 96.88%, surpassing CNN-based models in both computational efficiency and energy consumption—key factors for edge deployment.

To support the framework theoretically, four analyses were provided: a Lipschitz analysis establishing the SNN’s stability to input perturbations; a formalization of HDC operations and hyperspace definitions; several lemmas on Hamming distance preservation and statistical properties of high-dimensional spaces; and three theorems proving robustness to input noise, representational fidelity of Cluster-HVs, and resilience to hardware errors.

Overall, DSN reduces energy consumption by up to $35.33\times$ and parameters by about threefold compared to CNNs, highlighting its suitability for resource-constrained edge systems. Future work will further optimize DSN for practical deployment, extend it to other physiological signals, and explore integration with neuromorphic hardware for maximal efficiency.

Acknowledgements

The work was supported by the National Key R&D Program of China under Grant 2021ZD0201300, the Foundation for Outstanding Research Groups

of Hubei Province of China under Grant 2025AFA012, the 111 Project on Computational Intelligence and Intelligent Control under Grant B18024, the National Natural Science Foundation of China under Grants 62406118, and the Postdoctoral Project of Hubei Province under Grant Number 2024HBB-HCXA017.

References

- [1] N. Li, R. Zhou, B. Krishna, A. Pradhan, H. Lee, J. He, N. Jiang, Non-invasive Techniques for Muscle Fatigue Monitoring: A Comprehensive Survey, *ACM Computing Surveys* 56 (9) (2024) 1–40.
- [2] G. J. Rani, M. F. Hashmi, A. Gupta, Surface electromyography and artificial intelligence for human activity recognition-A systematic review on methods, emerging trends applications, challenges, and future implementation, *IEEE Access* (2023).
- [3] C.-K. Lin, A. Wild, G. N. Chinya, Y. Cao, M. Davies, D. M. Lavery, H. Wang, Programming spiking neural networks on Intel’s Loihi, *Computer* 51 (3) (2018) 52–61.
- [4] N. Rathi, I. Chakraborty, A. Kosta, A. Sengupta, A. Ankit, P. Panda, K. Roy, Exploring neuromorphic computing based on spiking neural networks: Algorithms to hardware, *ACM Computing Surveys* 55 (12) (2023) 1–49.
- [5] P. Kanerva, Hyperdimensional computing: An introduction to computing in distributed representation with high-dimensional random vectors, *Cognitive computation* 1 (2009) 139–159.
- [6] D. Kudithipudi, C. Schuman, C. M. Vineyard, T. Pandit, C. Merkel, R. Kubendran, J. B. Aimone, G. Orchard, C. Mayr, R. Benosman, et al., Neuromorphic computing at scale, *Nature* 637 (8047) (2025) 801–812.
- [7] D. B. Chaffin, Localized muscle fatigue—definition and measurement, *Journal of Occupational and Environmental Medicine* 15 (4) (1973) 346–354.
- [8] S. Rampichini, T. M. Vieira, P. Castiglioni, G. Merati, Complexity analysis of surface electromyography for assessing the myoelectric manifestation of muscle fatigue: A review, *Entropy* 22 (5) (2020) 529.

- [9] D. Roman-Liu, T. Tokarski, K. Wójcik, Quantitative assessment of upper limb muscle fatigue depending on the conditions of repetitive task load, *Journal of Electromyography and Kinesiology* 14 (6) (2004) 671–682.
- [10] J. Wang, S. Sun, Y. Sun, A muscle fatigue classification model based on LSTM and improved wavelet packet threshold, *Sensors* 21 (19) (2021) 6369.
- [11] X. Wang, X. Wang, C. Qiu, Q. Li, Muscle fatigue classification and the effect of electrical stimulation on muscle fatigue recovery, in: *Journal of Physics: Conference Series*, Vol. 1924, IOP Publishing, 2021, p. 012020.
- [12] Q. Liu, Y. Liu, C. Zhang, Z. Ruan, W. Meng, Y. Cai, Q. Ai, sEMG-Based Dynamic Muscle Fatigue Classification Using SVM With Improved Whale Optimization Algorithm, *IEEE Internet of Things Journal* 8 (23) (2021) 16835–16844. doi:10.1109/JIOT.2021.3056126.
- [13] D. Xiong, D. Zhang, X. Zhao, Y. Zhao, Deep Learning for EMG-based Human-Machine Interaction: A Review, *IEEE/CAA Journal of Automatica Sinica* 8 (3) (2021) 512–533. doi:10.1109/JAS.2021.1003865.
- [14] W. Zhong, X. Jiang, K. Szymaniak, M. Jabbari, C. Ma, K. Nazarpour, Deep Feature Learning from Electromyographic Signals for Gesture Recognition Systems, *IEEE Transactions on Neural Systems and Rehabilitation Engineering* (2025) 1–1doi:10.1109/TNSRE.2025.3635419.
- [15] J. Yoo, I. Yoo, I. Youn, S.-M. Kim, R. Yu, K. Kim, K. Kim, S.-B. Lee, Residual one-dimensional convolutional neural network for neuromuscular disorder classification from needle electromyography signals with explainability, *Computer Methods and Programs in Biomedicine* 226 (2022) 107079. doi:https://doi.org/10.1016/j.cmpb.2022.107079.
- [16] L. I. Barona López, F. M. Ferri, J. Zea, Ángel Leonardo Valdivieso Caraguay, M. E. Benalcázar, CNN-LSTM and post-processing for EMG-based hand gesture recognition, *Intelligent Systems with Applications* 22 (2024) 200352. doi:https://doi.org/10.1016/j.iswa.2024.200352.
- [17] A. A. Kareemulla, R. K. Sanodiya, A. C. Turlapaty, S. Naidu, EMGTTL: Transformers-Based Transfer Learning for Classification of ADL using Raw Surface EMG Signals, in: 2024 IEEE

Pune Section International Conference (PuneCon), 2024, pp. 1–6.
doi:10.1109/PuneCon63413.2024.10895386.

- [18] C. Lin, X. Zhang, C. Zhao, A parallel and efficient transformer deep learning network for continuous estimation of hand kinematics from electromyographic signals, *Scientific Reports* 15 (1) (2025) 36150. doi:10.1038/s41598-025-16268-y.
URL <https://doi.org/10.1038/s41598-025-16268-y>
- [19] J. Shin, A. S. M. Miah, S. Konnai, I. Takahashi, K. Hirooka, Hand gesture recognition using sEMG signals with a multi-stream time-varying feature enhancement approach, *Scientific Reports* 14 (1) (2024) 22061. doi:10.1038/s41598-024-72996-7.
URL <https://doi.org/10.1038/s41598-024-72996-7>
- [20] J. D. Nunes, M. Carvalho, D. Carneiro, J. S. Cardoso, Spiking neural networks: A survey, *IEEE access* 10 (2022) 60738–60764.
- [21] H. Wang, Y.-F. Li, K. Gryllias, Brain-inspired spiking neural networks for industrial fault diagnosis: A survey, challenges, and opportunities, *arXiv preprint arXiv:2401.02429* (2023).
- [22] W. Gerstner, W. M. Kistler, *Spiking neuron models: Single neurons, populations, plasticity*, Cambridge university press, 2002.
- [23] M. Heddes, I. Nunes, T. Givargis, A. Nicolau, A. Veidenbaum, Hyper-dimensional computing: a framework for stochastic computation and symbolic AI, *Journal of Big Data* 11 (1) (2024) 145.
- [24] H. M. Qassim, W. Z. W. Hasan, H. R. Ramli, H. H. Harith, L. N. I. Mat, L. I. Ismail, Proposed fatigue index for the objective detection of muscle fatigue using surface electromyography and a double-step binary classifier, *Sensors* 22 (5) (2022) 1900.
- [25] D. C. Toledo-Pérez, J. Rodriguez-Resendiz, R. A. Gomez-Loenzo, A study of computing zero crossing methods and an improved proposal for EMG signals, *IEEE Access* 8 (2020) 8783–8790.
- [26] Ü. Kaljumäe, O. Hänninen, O. Airaksinen, Knee extensor fatigability and strength after bicycle ergometer training, *Archives of physical medicine and rehabilitation* 75 (5) (1994) 564–567.

- [27] A. Phinyomark, R. N. Khushaba, E. Scheme, Feature extraction and selection for myoelectric control based on wearable EMG sensors, *Sensors* 18 (5) (2018) 1615.
- [28] S. M. Cerqueira, R. Vilas Boas, J. Figueiredo, C. P. Santos, A Comprehensive Dataset of Surface Electromyography and Self-Perceived Fatigue Levels for Muscle Fatigue Analysis, *Sensors* 24 (24) (2024) 8081.
- [29] I. Melzer, N. Benjuya, J. Kaplanski, Age related changes in muscle strength and fatigue, *Isokinetics and exercise science* 8 (2) (2000) 73–83.
- [30] L. A. Wojcik, M. A. Nussbaum, D. Lin, P. A. Shibata, M. L. Madigan, Age and gender moderate the effects of localized muscle fatigue on lower extremity joint torques used during quiet stance, *Human movement science* 30 (3) (2011) 574–583.
- [31] Y. Wang, C. Lu, M. Zhang, J. Wu, Z. Tang, Research on the Recognition of Various Muscle Fatigue States in Resistance Strength Training, in: *Healthcare*, Vol. 10, MDPI, 2022, p. 2292.
- [32] Q. Liu, Y. Liu, C. Zhang, Z. Ruan, W. Meng, Y. Cai, Q. Ai, sEMG-based dynamic muscle fatigue classification using SVM with improved whale optimization algorithm, *IEEE Internet of Things Journal* 8 (23) (2021) 16835–16844.
- [33] N. D. Alotaibi, H. Jahanshahi, Q. Yao, J. Mou, S. Bekiros, An ensemble of long short-term memory networks with an attention mechanism for upper limb electromyography signal classification, *Mathematics* 11 (18) (2023) 4004.
- [34] P. Tsinganos, B. Cornelis, J. Cornelis, B. Jansen, A. Skodras, Improved Gesture Recognition Based on sEMG Signals and TCN, in: *ICASSP 2019 - 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2019, pp. 1169–1173. doi:10.1109/ICASSP.2019.8683239.
- [35] M. Montazerin, S. Zabihi, E. Rahimian, A. Mohammadi, F. Naderkhani, ViT-HGR: Vision Transformer-based Hand Gesture Recognition from High Density Surface EMG Signals, *CoRR* abs/2201.10060 (2022). arXiv:2201.10060.
URL <https://arxiv.org/abs/2201.10060>

- [36] M. A. Scrugli, G. Leone, P. Busia, L. Raffo, P. Meloni, Real-time senn processing with spiking neural networks on a low-power 5k-lut fpga, *IEEE Transactions on Biomedical Circuits and Systems* 19 (1) (2025) 68–81. doi:10.1109/TBCAS.2024.3456552.

Appendix A. Notations and Definitions

Hyperspace: The hyperspace $\mathcal{H}^D$ consists of all bipolar hypervectors of dimension D , where each entry takes a value from the set $\{+1, -1\}$. Formally, we define:

$$\mathcal{H}^D := \{-1, +1\}^D = \{A \in \mathbb{R}^D \mid a_i \in \{+1, -1\}, \forall i = 1, \dots, D\} \quad (\text{A.1})$$

with a_i representing the i -th component of hypervector A .

Unless stated otherwise, for a *random hypervector* $A \in \mathcal{H}^D$, each component a_i is independently sampled from a symmetric binary distribution:

$$\mathbb{P}(a_i = +1) = \mathbb{P}(a_i = -1) = 0.5, \quad i = 1, \dots, D. \quad (\text{A.2})$$

Elementary Functions: For any hypervectors $A, B \in \mathcal{H}^D$, we introduce the following basic operations:

- Count of $+1$ entries in A :

$$\text{pos}(A) := \sum_{i=1}^D \mathbb{I}(a_i = +1) \quad (\text{A.3})$$

- Count of -1 entries in A :

$$\text{neg}(A) := \sum_{i=1}^D \mathbb{I}(a_i = -1) \quad (\text{A.4})$$

- Binding (element-wise multiplication) of A and B :

$$A \otimes B := A \times B \quad (\text{A.5})$$

- Bundling (element-wise majority) of A and B :

$$A \oplus B := [A + B] \quad (\text{A.6})$$

Here, $\mathbb{I}(\cdot)$ is the indicator function, $\times$ denotes element-wise multiplication, $+$ is element-wise addition, and $[\cdot]$ represents the element-wise majority function, which outputs $+1$ if the sum is positive, -1 if negative, and randomly selects between $\{-1, +1\}$ when the sum is zero.

Hamming Distance: For two hypervectors $A, B \in \mathcal{H}^D$, the Hamming distance measures the fraction of positions where they differ:

$$d_H : \mathcal{H}^D \times \mathcal{H}^D \rightarrow [0, 1], \quad (\text{A.7})$$

$$d_H(A, B) = \frac{1}{D} \sum_{i=1}^D \mathbb{I}(a_i \neq b_i) = \frac{1}{D} \text{neg}(A \otimes B). \quad (\text{A.8})$$

Level Set: A set of M hypervectors $\mathcal{L} = \{L^{(j)}\}_{j=1}^M \subset \mathcal{H}^D$ is called a level set if it is constructed as follows:

- Sample a base hypervector $L^{(1)} \in \mathcal{H}^D$ randomly.
- Initialize a set $\mathcal{B} = \emptyset$ to track the flipped positions.
- For $i = 2, \dots, M$:
 - Randomly select $\frac{D}{M-1}$ positions from $\{1, \dots, D\} \setminus \mathcal{B}$ (positions not yet flipped). Assume $\frac{D}{M-1} \in \mathbb{N}$ for simplicity.
 - Flip the selected bits in $L^{(i-1)}$ to generate $L^{(i)}$.
 - Update $\mathcal{B}$ to include the newly flipped positions.

Under this construction, the Hamming distance between any two hypervectors $L^{(i)}$ and $L^{(j)}$ is

$$d_H(L^{(i)}, L^{(j)}) = \frac{|i - j|}{M - 1}. \quad (\text{A.9})$$

Mapping with Levels: Let $\mathcal{X} \subseteq \mathbb{R}^d$ denote the feature space. Consider d random hypervectors $\mathcal{ID} = \{ID^{(1)}, \dots, ID^{(d)}\} \subset \mathcal{H}^D$ and a level set $\mathcal{L} = \{L^{(1)}, \dots, L^{(M)}\} \subset \mathcal{H}^D$.

For each feature dimension $n = 1, \dots, d$, define thresholds $\theta_{n,1} < \dots < \theta_{n,M-1}$ to partition $\mathbb{R}$ into M intervals:

$$I_0 = (-\infty, \theta_{n,1}), \quad I_1 = [\theta_{n,1}, \theta_{n,2}), \dots, I_{M-1} = [\theta_{n,M-1}, +\infty) \quad (\text{A.10})$$

Given a feature value $x_i[n]$, assign it to the unique interval I_m such that $x_i[n] \in I_m$, defining

$$l_n(x) = \begin{cases} 1, & x < \theta_{n,\alpha} \\ \lfloor \frac{x - \theta_{n,\alpha}}{\theta_{n,\beta} - \theta_{n,\alpha}} \cdot M + 1 \rfloor, & x \in [\theta_{n,\alpha}, \theta_{n,\beta}) \\ M, & x \geq \theta_{n,\beta} \end{cases} \quad (\text{A.11})$$

where $\theta_{n,\alpha}$ and $\theta_{n,\beta}$ are the 2% and 98% quantiles of feature n .

Appendix B. Lemmas and Propositions

Lemma 1 (Hamming Distance between product and multiplier).

For hypervectors $A, B \in \mathcal{H}^D$, the Hamming distance between A and $A \times B$ depends only on B :

$$d_H(A, A \times B) = \frac{1}{D} \text{neg}(B). \quad (\text{B.1})$$

Proof. By the definition of element-wise multiplication, the i -th component satisfies

$$a_i b_i = \begin{cases} a_i, & b_i = +1, \\ -a_i, & b_i = -1, \end{cases} \quad (\text{B.2})$$

and equivalently,

$$a_i \neq a_i b_i \iff b_i = -1. \quad (\text{B.3})$$

Thus, the Hamming distance can be written as

$$d_H(A, A \times B) = \frac{1}{D} \sum_{i=1}^D \mathbb{I}(a_i \neq a_i b_i) = \frac{1}{D} \sum_{i=1}^D \mathbb{I}(b_i = -1) = \frac{1}{D} \text{neg}(B). \quad (\text{B.4})$$

Lemma 2 (Hamming Distance Preservation under Multiplication). For hypervectors $A, B, C \in \mathcal{H}^D$:

$$d_H(A \times B, B \times C) = d_H(A, C). \quad (\text{B.5})$$

Proof. Considering component-wise multiplication:

$$a_i b_i \neq b_i c_i \iff a_i \neq c_i, \quad (\text{B.6})$$

and therefore

$$d_H(A \times B, B \times C) = \frac{1}{D} \sum_{i=1}^D \mathbb{I}(a_i b_i \neq b_i c_i) = \frac{1}{D} \sum_{i=1}^D \mathbb{I}(a_i \neq c_i) = d_H(A, C). \quad (\text{B.7})$$

Lemma 3 (Hamming distance between two random hypervectors). Let $A, B \in \mathcal{H}^D$ be independent random hypervectors. Then, for any $\epsilon > 0$,

$$\lim_{D \rightarrow \infty} \mathbb{P}(|d_H(A, B) - 0.5| > \epsilon) = 0. \quad (\text{B.8})$$

Proof. Define indicator variables $X_i := \mathbb{I}(a_i \neq b_i)$, so that

$$d_H(A, B) = \frac{1}{D} \sum_{i=1}^D X_i. \quad (\text{B.9})$$

Since each a_i and b_i is i.i.d. from $\{-1, +1\}$, each X_i is an independent Bernoulli variable with mean 0.5. By the Weak Law of Large Numbers:

$$\lim_{D \rightarrow \infty} \mathbb{P}\left(\left|\frac{1}{D} \sum_{i=1}^D X_i - 0.5\right| > \epsilon\right) = 0, \quad (\text{B.10})$$

so the Hamming distance concentrates around 0.5.

Lemma 4 (Hamming distance between a random hypervector and its product with another). Let $A, B \in \mathcal{H}^D$ be independent random hypervectors. Then, for any $\epsilon > 0$,

$$\lim_{D \rightarrow \infty} \mathbb{P}(|d_H(A, A \times B) - 0.5| > \epsilon) = 0. \quad (\text{B.11})$$

Proof. From Lemma 1, we have

$$d_H(A, A \times B) = \frac{\text{neg}(B)}{D}. \quad (\text{B.12})$$

Since B is random with each component i.i.d., applying the Weak Law of Large Numbers gives

$$\lim_{D \rightarrow \infty} \mathbb{P}\left(\left|\frac{\text{neg}(B)}{D} - 0.5\right| > \epsilon\right) = 0, \quad (\text{B.13})$$

which implies the result.

Proposition (Statistical view of Hamming Distance). Consider hypervectors $A, B \in \mathcal{H}^D$ with

$$d_H(A, B) = \delta, \quad (\text{B.14})$$

and define

$$Z_i = \mathbb{I}(a_i \neq b_i) \quad (\text{B.15})$$

to indicate whether the i -th bit differs. For large D , we can approximate

$$Z_i \sim_{i.i.d.} \text{Bernoulli}(\delta), \quad i = 1, \dots, D. \quad (\text{B.16})$$

Proof. By definition,

$$\frac{1}{D} \sum_{i=1}^D Z_i = d_H(A, B) = \delta. \quad (\text{B.17})$$

Although the Z_i are not strictly independent (their sum is fixed), for large D :

- The probability that the empirical mean deviates from δ is negligible:

$$\mathbb{P}\left(\left|\frac{1}{D} \sum_{i=1}^D Z_i - \delta\right| > \epsilon\right) \leq 2 \exp(-2D\epsilon^2). \quad (\text{B.18})$$

- If A and B are generated randomly conditioned on their Hamming distance, the mismatch positions are uniformly random.

Hence, Z_i can be treated as i.i.d. Bernoulli variables for large D .